PROMPT LIFE V0.9.0

Lesson Cards

One accessible curriculum-review card per lesson, built from current app data plus inventory notes and rubric scores.

What Is an LLM?

A learned next-token prediction system

Stage: architecture | Priority: low | Rubric avg: 4.4/5

DEFINITION
A large language model is a learned system that turns current context into probabilities for the next token.

CORE EXPLANATION
An LLM does not read a prompt and write a whole answer at once. During inference, the current context enters the model, learned weights shape internal hidden states, the model scores possible next tokens, one token is chosen, and that token is appended before the next run. Training built the weights earlier; inference uses them.

WHERE IT HAPPENS
This card describes the whole model lifecycle, but especially normal inference: current context to next-token probabilities to generated response token.

DURABLE VS TEMPORARY
The weights were changed during training or fine-tuning. During ordinary inference, hidden states are temporary and weights normally stay fixed.

PROMPT VS RESPONSE
Prompt tokens are given to the model. Response tokens are generated one at a time.

WHY IT MATTERS
This definition lowers the mystery. The model can be powerful without being a mind, database, or hand-written rulebook.

RELATIONSHIP
Next, we compare the older rule-based dream of AI with the deep-learning approach that made LLMs work.

METAPHOR
A vast autocomplete engine with learned structure.

BRAIN METAPHOR
Like a person using context to anticipate what word might come next.

BRAIN LIMIT
A person has a body, goals, lived experience, and awareness. The model has learned weights and temporary computations.

MISCONCEPTION
An LLM is a conscious reader or a database.

VISUAL AID
`llm-overview`

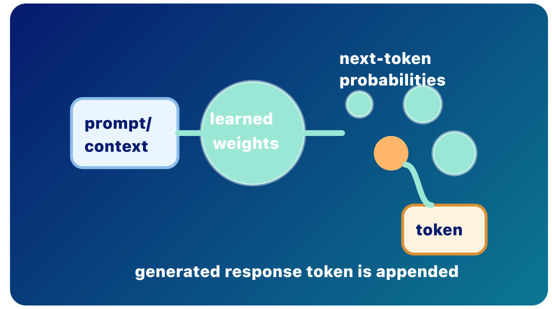
CURRENT EXERCISE
`pick-next-token`: Pick the Next Token

CHECKPOINT
Which description is most accurate? Correct: A learned prediction system. Incorrect: A conscious reader; A hand-written rulebook; A search engine that knows every file.

FEEDBACK
An LLM uses learned weights to turn context into next-token probabilities. Choice notes: A conscious reader: Not quite. The model is powerful, but it is not conscious, not a database, and not a hand-coded rulebook. A hand-written rulebook: Not quite. The model is powerful, but it is not conscious, not a database, and not a hand-coded rulebook. A search engine that knows every file: Not quite. The model is powerful, but it is not conscious, not a database, and not a hand-coded rulebook.

REWRITE NOTES
Risk: An LLM is a conscious reader or a database.
Missing: An LLM does not read a prompt and write a whole answer at once. During inference, the current context enters the model, learned weights shape internal hidden states, the model scores possible next tokens, one token is chosen, and that token is appended before the next run. Training built the weights earlier; inference uses them.

ILLUSTRATION NEEDED
Current visual aid: llm-overview. One-page prompt lifecycle: prompt enters, fixed weights process, next token sampled, response grows.



Prompt to Prediction
Prompt/context flows through learned weights into next-token probabilities; one generated response token is appended.

1. Prompt/context is given input.
2. Learned weights are used during inference.
3. The selected response token is appended.

Rubric Scores

Accuracy 5/5	Placement 4/5	Relationship 5/5	Prompt/response 5/5	Durable/temp 5/5
Metaphor 4/5	Brain/cognition 4/5	Visual 4/5	Exercise 4/5	Mobile 4/5

llm · prompt · response · token · weight · inference

Rationalists vs Empiricists

Rules versus learned patterns

Stage: architecture | Priority: medium | Rubric avg: 3.6/5

DEFINITION
Symbolic AI tries to reason with explicit rules and symbols; deep learning learns useful patterns from data by adjusting weights.

CORE EXPLANATION
Early AI often followed the rationalist dream: intelligence as explicit rules, logic, symbols, and search. Deep learning follows a more empiricist path: expose a model to many examples, measure error, and let the system learn internal representations that work. LLMs mostly belong to the deep-learning tradition, although real systems often combine learned models with rules, tools, retrieval, filters, and policies.

WHERE IT HAPPENS
This is historical and architectural background. It explains what kind of system an LLM is.

DURABLE VS TEMPORARY
Deep learning changes weights during training. Symbolic rules are usually written or configured explicitly.

PROMPT VS RESPONSE
A prompt is not matched against one giant rulebook. It is processed through learned parameters that shape response-token probabilities.

WHY IT MATTERS
This helps learners stop expecting LLMs to behave like ordinary hand-written programs.

RELATIONSHIP
If LLMs are learned systems, then the next question is: how are they learned? That leads to training.

METAPHOR
A rulebook versus a trained landscape.

BRAIN METAPHOR
Humans use both explicit reasoning and learned intuition.

BRAIN LIMIT
An LLM is not a full human mind combining logic, experience, emotion, and embodiment. It is a learned prediction system.

MISCONCEPTION
LLMs are hand-coded rulebooks, or symbolic AI is obsolete.

VISUAL AID
`traditions`

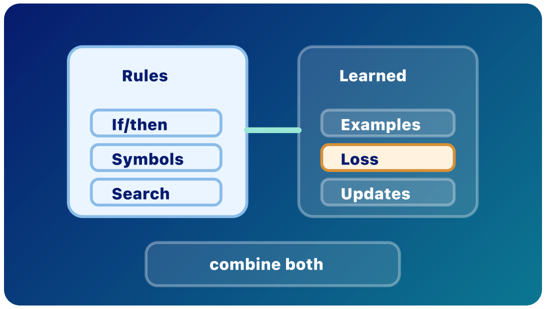
CURRENT EXERCISE
`durable-or-temporary`: Durable or Temporary?

CHECKPOINT
Which phrase best describes the LLM tradition? Correct: Learned patterns from data. Incorrect: Only fixed if-then rules; A spreadsheet macro; A conscious reasoning engine.

FEEDBACK
LLMs are learned prediction systems shaped by data. Choice notes: Only fixed if-then rules: Not quite. Rules still matter in software, but LLM fluency mostly comes from learned weights. A spreadsheet macro: Not quite. Rules still matter in software, but LLM fluency mostly comes from learned weights. A conscious reasoning engine: Not quite. Rules still matter in software, but LLM fluency mostly comes from learned weights.

REWRITE NOTES
Risk: LLMs are hand-coded rulebooks, or symbolic AI is obsolete. **Missing:** Early AI often followed the rationalist dream: intelligence as explicit rules, logic, symbols, and search. Deep learning follows a more empiricist path: expose a model to many examples, measure error, and let the system learn internal representations that work. LLMs mostly belong to the deep-learning tradition, although real systems often combine learned models with rules, tools, retrieval, filters, and policies.

ILLUSTRATION NEEDED
Current visual aid: traditions. Split-panel rulebook versus learned landscape, with a small note that systems can combine them.



Rules and Learned Patterns
Symbolic/rationalist systems use explicit rules; deep-learning/empiricist systems learn useful patterns from examples.

1. Rules: if/then logic and symbols.
2. Examples: data, loss, and weight updates.
3. Modern AI products can combine learned models with rules and tools.

Rubric Scores

Accuracy 5/5	**Placement** 4/5	**Relationship** 4/5	**Prompt/response** 3/5	**Durable/temp** 3/5
Metaphor 4/5	**Brain/cognition** 4/5	**Visual** 3/5	**Exercise** 2/5	**Mobile** 4/5

rationalism · empiricism · symbolic-ai · deep-learning · training · weight

Training

Prediction error updates weights

Stage: pretraining | Priority: low | Rubric avg: 4.3/5

DEFINITION
Training is the process that adjusts model weights by comparing predictions with target answers, such as actual next tokens.

CORE EXPLANATION
During training, the model sees examples, predicts something, measures how wrong it was, and updates its weights to do slightly better next time. This happens over many examples and many updates. Training is where durable model capability is created.

WHERE IT HAPPENS
Before ordinary use, in a separate optimization process.

DURABLE VS TEMPORARY
Training durably changes weights. Ordinary inference does not.

PROMPT VS RESPONSE
Training examples may look like text prompts and completions, but the purpose is different: the system is updating weights, not merely producing a live response for a user.

WHY IT MATTERS
This is the dividing line between learning and using. Many AI myths come from confusing training with inference.

RELATIONSHIP
Pretraining is the broad first version of this training loop.

METAPHOR
Practice that changes the instrument before the performance.

BRAIN METAPHOR
Like practice shaping future habits or skill.

BRAIN LIMIT
The model is not reflecting on mistakes. An optimizer changes numbers to reduce error.

MISCONCEPTION
When I chat with the model, it automatically trains itself.

VISUAL AID
`training-loop`

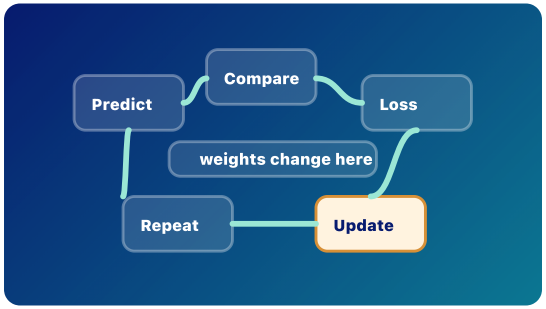
CURRENT EXERCISE
`training-nudge`: Training Nudge

CHECKPOINT
What makes training different from inference? Correct: Training durably updates weights. Incorrect: Training only reads the current context window; Training is the same as sampling; Training writes the final answer.

FEEDBACK
Training changes the model; inference uses the model. Choice notes: Training only reads the current context window: Not quite. Training is a weight-changing process, not ordinary response generation. Training is the same as sampling: Not quite. Training is a weight-changing process, not ordinary response generation. Training writes the final answer: Not quite. Training is a weight-changing process, not ordinary response generation.

REWRITE NOTES
Risk: When I chat with the model, it automatically trains itself. **Missing:** During training, the model sees examples, predicts something, measures how wrong it was, and updates its weights to do slightly better next time. This happens over many examples and many updates. Training is where durable model capability is created.

ILLUSTRATION NEEDED
Current visual aid: training-loop. Training loop with predict, compare, loss, update weights, repeat.



Training Loop
Predict, compare, loss, update weights, repeat. Training changes weights.

1. Predict and compare against a target.
2. Loss measures error.
3. Weight updates make training durable.

Rubric Scores

Accuracy 5/5	**Placement** 5/5	**Relationship** 4/5	**Prompt/response** 4/5	**Durable/temp** 5/5
Metaphor 4/5	**Brain/cognition** 4/5	**Visual** 4/5	**Exercise** 4/5	**Mobile** 4/5

training | loss | weight | weight-update | parameter | training-data | inference | forward pass

Pretraining

Broad learning before normal use

Stage: pretraining · Priority: low · Rubric avg: 4.1/5

DEFINITION
Pretraining is the broad first training phase where a model learns general patterns by repeatedly predicting tokens across large datasets.

CORE EXPLANATION
Before a model can be useful in ordinary conversation, it is pretrained on large collections of text and other data. The model repeatedly predicts tokens, measures error, and updates weights. Over time it learns grammar, styles, facts, associations, reasoning patterns, and task shapes. This does not mean it stores every source as a perfect memory.

WHERE IT HAPPENS
Before deployment, during the broad foundation-building stage.

DURABLE VS TEMPORARY
Pretraining durably changes weights.

PROMPT VS RESPONSE
Pretraining creates the weights later used to process prompts and generate responses.

WHY IT MATTERS
Pretraining is why next-token prediction can become surprisingly capable. It installs broad pattern knowledge into weights.

RELATIONSHIP
Pretraining builds broad capability. Overfitting explains a failure mode: learning examples too narrowly instead of patterns that generalize.

METAPHOR
Billions of tiny raindrops carving paths through a landscape.

BRAIN METAPHOR
Like broad education shaping later intuition.

BRAIN LIMIT
Humans connect learning to lived experience. A model stores statistical structure in parameters.

MISCONCEPTION
Pretraining means the model can perfectly recall everything it saw.

VISUAL AID
`pretraining-rain`

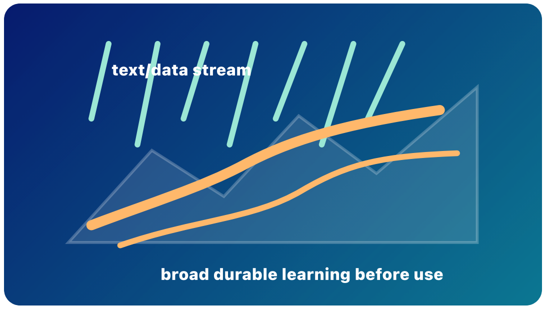
CURRENT EXERCISE
`training-nudge`: Training Nudge

CHECKPOINT
During pretraining, what changes? Correct: Model weights. Incorrect: Only the current chat; Only the user interface; Nothing changes.

FEEDBACK
Pretraining changes weights before normal use. Choice notes: Only the current chat: Not quite. Pretraining is a durable training process, not a temporary chat event. Only the user interface: Not quite. Pretraining is a durable training process, not a temporary chat event. Nothing changes: Not quite. Pretraining is a durable training process, not a temporary chat event.

REWRITE NOTES
Risk: Pretraining means the model can perfectly recall everything it saw. **Missing:** Before a model can be useful in ordinary conversation, it is pretrained on large collections of text and other data. The model repeatedly predicts tokens, measures error, and updates weights. Over time it learns grammar, styles, facts, associations, reasoning patterns, and task shapes. This does not mean it stores every source as a perfect memory.

ILLUSTRATION NEEDED
Current visual aid: pretraining-rain. Broad data rain carving durable pathways into a model landscape.



Broad Pretraining
A wide stream of data shapes broad durable learning before normal use.

1. Many examples flow through the training loop.
2. Repeated updates shape broad model capability.

Rubric Scores

Accuracy 5/5	**Placement** 5/5	**Relationship** 4/5	**Prompt/response** 4/5	**Durable/temp** 5/5
Metaphor 4/5	**Brain/cognition** 4/5	**Visual** 4/5	**Exercise** 2/5	**Mobile** 4/5

pretraining · training · training-data · weight · loss · next-token · inference

Overfitting and Generalization

Memorizing examples versus learning patterns

Stage: pretraining | Priority: high | Rubric avg: 4.4/5

DEFINITION
Overfitting happens when a model learns training examples too narrowly and performs poorly on new examples; generalization means learned patterns work beyond the examples seen during training.

CORE EXPLANATION
A model can fit old examples very well but still fail on new cases. That is overfitting. Good learning captures patterns that transfer. For LLMs, generalization is why a model can answer prompts it has never seen exactly before. Overfitting is why evaluation needs unseen examples, not just training performance.

WHERE IT HAPPENS
During training, evaluation, and model selection.

DURABLE VS TEMPORARY
Overfitting is about what gets installed into weights during training. It is not a temporary inference effect.

PROMPT VS RESPONSE
A model that generalizes well can handle new prompts. A model that overfits may repeat brittle patterns from training.

WHY IT MATTERS
This helps learners see why more memorization is not the same as better intelligence.

RELATIONSHIP
Pretraining should build broad patterns. Fine-tuning must adapt behavior without making the model too narrow.

METAPHOR
Studying only the answer key versus learning the subject.

BRAIN METAPHOR
Like memorizing practice questions without understanding the principle.

BRAIN LIMIT
The model is not consciously memorizing. Optimization can make it rely on brittle numerical patterns.

MISCONCEPTION
If a model performs well on training data, it has learned well.

VISUAL AID
`overfitting-generalization`

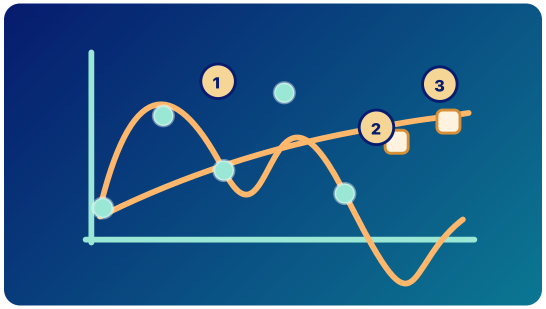
CURRENT EXERCISE
`durable-or-temporary`: Durable or Temporary?

CHECKPOINT
Which is a sign of overfitting? Correct: Great performance on training examples but poor performance on new examples. Incorrect: A model using a context window; A model retrieving documents with RAG; A model generating one token at a time.

FEEDBACK
Generalization matters more than memorizing the answer key. Choice notes: A model using a context window: Not quite. Overfitting is a training/evaluation failure mode, not context or retrieval. A model retrieving documents with RAG: Not quite. Overfitting is a training/evaluation failure mode, not context or retrieval. A model generating one token at a time: Not quite. Overfitting is a training/evaluation failure mode, not context or retrieval.

REWRITE NOTES
Risk: If a model performs well on training data, it has learned well. **Missing:** A model can fit old examples very well but still fail on new cases. That is overfitting. Good learning captures patterns that transfer. For LLMs, generalization is why a model can answer prompts it has never seen exactly before. Overfitting is why evaluation needs unseen examples, not just training performance.

ILLUSTRATION NEEDED
Current visual aid: overfitting-generalization. Two curves: an overfit curve that traces training dots too tightly and a smoother curve that predicts new dots.



Overfitting vs Generalization
Memorizing examples is not the same as learning transferable patterns.

1. Overfit curve traces old dots too tightly.
2. Generalizing curve is smoother and reaches new examples.
3. Held-out examples test whether learning transfers.

Rubric Scores

Accuracy 5/5	Placement 5/5	Relationship 4/5	Prompt/response 4/5	Durable/temp 5/5
Metaphor 4/5	Brain/cognition 4/5	Visual 5/5	Exercise 4/5	Mobile 4/5

overfitting · generalization · validation-data · training-data · evaluation · training

Fine-Tuning

Targeted shaping after pretraining

Stage: fine-tuning | Priority: low | Rubric avg: 4.1/5

DEFINITION
Fine-tuning is additional training that adapts a pretrained model toward a task, domain, style, or preference.

CORE EXPLANATION
A pretrained model knows broad language patterns, but it may not behave the way users need. Fine-tuning adds targeted examples or preference data so future responses are more helpful, specialized, safer, or better aligned with a task. Fine-tuning can update model weights directly or add smaller adapter weights.

WHERE IT HAPPENS
After pretraining and before or between deployment cycles.

DURABLE VS TEMPORARY
Fine-tuning can durably change weights or add durable adapter behavior. It is different from a prompt, which usually only steers the current run.

PROMPT VS RESPONSE
Fine-tuning shapes future response patterns. It does not generate the current response token by token.

WHY IT MATTERS
This explains how one base model can become many differently behaving assistants.

RELATIONSHIP
Fine-tuning adapts a model; alignment is one major reason to fine-tune or otherwise shape behavior.

METAPHOR
Carving useful trails through an already huge terrain.

BRAIN METAPHOR
Like coaching someone to follow a house style or professional norm.

BRAIN LIMIT
The model is not choosing a new identity or values. Training changes output patterns.

MISCONCEPTION
Prompting once is the same as fine-tuning.

VISUAL AID
`fine-tune-path`

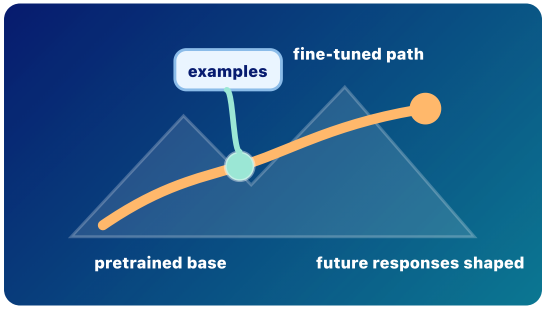
CURRENT EXERCISE
`durable-or-temporary`: Durable or Temporary?

CHECKPOINT
Which action is closest to fine-tuning? Correct: Additional training that shapes future responses. Incorrect: Typing one better prompt; Retrieving one PDF into context; Sampling the next token.

FEEDBACK
Fine-tuning changes future behavior more durably than a single prompt. Choice notes: Typing one better prompt: Not quite. Prompting and RAG steer the current context; fine-tuning changes model behavior through training. Retrieving one PDF into context: Not quite. Prompting and RAG steer the current context; fine-tuning changes model behavior through training. Sampling the next token: Not quite. Prompting and RAG steer the current context; fine-tuning changes model behavior through training.

REWRITE NOTES
Risk: Prompting once is the same as fine-tuning.
Missing: A pretrained model knows broad language patterns, but it may not behave the way users need. Fine-tuning adds targeted examples or preference data so future responses are more helpful, specialized, safer, or better aligned with a task. Fine-tuning can update model weights directly or add smaller adapter weights.

ILLUSTRATION NEEDED
Current visual aid: fine-tune-path. Targeted trail added onto an existing pretrained terrain.



Fine-Tuning Path
Targeted examples nudge an already-trained model toward a desired pattern.

1. Fine-tuning starts from a pretrained base.
2. Targeted examples shape future responses.

Rubric Scores

Accuracy 5/5	Placement 5/5	Relationship 4/5	Prompt/response 4/5	Durable/temp 5/5
Metaphor 4/5	Brain/cognition 4/5	Visual 4/5	Exercise 2/5	Mobile 4/5

fine-tuning · adapter · pretraining · training · prompt · response · alignment

Alignment

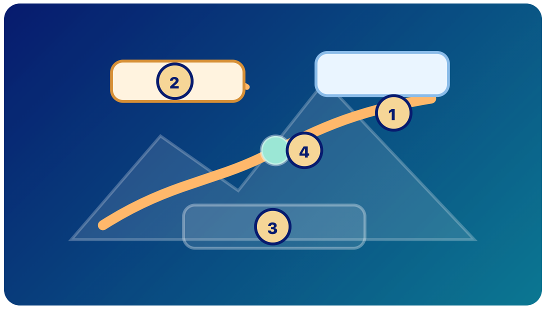
Shaping behavior toward human goals

Stage: alignment | Priority: high | Rubric avg: 4.5/5

DEFINITION
Alignment is the effort to shape AI behavior toward human instructions, preferences, safety boundaries, and values.

CORE EXPLANATION
A model trained only to predict likely text may produce outputs that are fluent but unhelpful, unsafe, biased, or misleading. Alignment uses methods such as instruction tuning, human feedback, preference optimization, policies, filters, evaluations, and system design to shape model behavior. Alignment is not one switch. It is an ongoing effort to make model behavior more useful, reliable, and compatible with human goals.

WHERE IT HAPPENS
During fine-tuning, human feedback learning, system design, evaluation, deployment, and policy enforcement.



Alignment Landscape
Alignment shapes behavior with preferred paths, guardrails, feedback, and policies; it does not create moral agency.

1. Preferred path: behavior the system encourages.
2. Warning zone: outputs to avoid or handle carefully.
3. Policy check: runtime or review constraint.
4. Feedback: behavior signal, not conscience.

DURABLE VS TEMPORARY
Some alignment work durably changes weights or adapter weights. Some alignment happens through system prompts, filters, retrieval, tool rules, or runtime safeguards that steer behavior temporarily.

PROMPT VS RESPONSE
Alignment affects how the model responds to prompts, but it does not mean the model understands morality like a person.

WHY IT MATTERS
Alignment is why a model may refuse harmful requests, follow instructions, or prefer clearer helpful answers over merely likely text.

RELATIONSHIP
Fine-tuning can shape model behavior. Alignment asks: toward what goals and constraints should that behavior be shaped?

METAPHOR
Guardrails, coaching, and a compass.

BRAIN METAPHOR
Like learning social norms or professional expectations.

BRAIN LIMIT
The model does not acquire human values, moral agency, conscience, or care. Its behavior is shaped by training and system constraints.

MISCONCEPTION
Aligned means morally good or fully trustworthy.

VISUAL AID
`alignment`

CURRENT EXERCISE
`durable-or-temporary`: Durable or Temporary?

CHECKPOINT
What does alignment try to do? Correct: Shape model behavior toward human goals, instructions, and safety boundaries. Incorrect: Make the model conscious; Guarantee every answer is true; Permanently solve all AI risk.

FEEDBACK
Alignment shapes behavior, but it is not magic morality. Choice notes: Make the model conscious: Not quite. Alignment is a set of technical and human processes, not consciousness or a perfect safety guarantee. Guarantee every answer is true: Not quite. Alignment is a set of technical and human processes, not consciousness or a perfect safety guarantee. Permanently solve all AI risk: Not quite. Alignment is a set of technical and human processes, not consciousness or a perfect safety guarantee.

REWRITE NOTES
Risk: Aligned means morally good or fully trustworthy.
Missing: A model trained only to predict likely text may produce outputs that are fluent but unhelpful, unsafe, biased, or misleading. Alignment uses methods such as instruction tuning, human feedback, preference optimization, policies, filters, evaluations, and system design to shape model behavior. Alignment is not one switch. It is an ongoing effort to make model behavior more useful, reliable, and compatible with human goals.

ILLUSTRATION NEEDED
Current visual aid: alignment. Response landscape with preferred paths, warning zones, guardrails, policy checkpoint, and human feedback marker.

Rubric Scores

Accuracy 5/5	Placement 5/5	Relationship 4/5	Prompt/response 4/5	Durable/temp 5/5
Metaphor 4/5	Brain/cognition 5/5	Visual 5/5	Exercise 4/5	Mobile 4/5

alignment · instruction-tuning · human-feedback-learning · rlhf · preference-optimization · policy · guardrail · evaluation · fine-tuning

Inference

A forward pass, not a memory update

Stage: inference | Priority: low | Rubric avg: 4.4/5

DEFINITION
Inference is normal model use: fixed learned weights process the current context to produce scores for the next response token.

CORE EXPLANATION
During ordinary use, the current context enters the model, embeddings and hidden states are computed, the final hidden state produces next-token scores, and decoding chooses one response token. Then the token is appended and the process repeats. The learned weights are used, but they normally stay fixed.

WHERE IT HAPPENS
Every time a deployed model answers a prompt.

DURABLE VS TEMPORARY
Inference creates temporary hidden states and next-token scores for this run. It normally does not durably change the model weights.

PROMPT VS RESPONSE
Prompt tokens, retrieved context, conversation history, and response-so-far tokens can all be current context; inference generates the next response token.

WHY IT MATTERS
This is the line between using a trained model and training a model.

RELATIONSHIP
Batch 1 explained how weights are shaped before use. Inference is the live run that uses those weights without rewriting them.

METAPHOR
Using a map to choose a route, not redrawing the map.

BRAIN METAPHOR
Like using working memory to answer from the information currently in front of you.

BRAIN LIMIT
The model is not conscious or living; it computes temporary vectors with fixed weights.

MISCONCEPTION
Normal chat automatically trains the model.

VISUAL AID
`inference-pass`

CURRENT EXERCISE
`training-nudge`: Training Nudge

CHECKPOINT
What changes during ordinary inference? Correct: Temporary hidden states. Incorrect: The model weights permanently; The training dataset; Every document the model has seen.

FEEDBACK
Inference uses fixed weights and creates temporary states for this run. Choice notes: The model weights permanently: Not quite. Ordinary inference normally does not rewrite weights or training data. The training dataset: Not quite. Ordinary inference normally does not rewrite weights or training data. Every document the model has seen: Not quite. Ordinary inference normally does not rewrite weights or training data.

REWRITE NOTES
Risk: Normal chat automatically trains the model.
Missing: During ordinary use, the current context enters the model, embeddings and hidden states are computed, the final hidden state produces next-token scores, and decoding chooses one response token. Then the token is appended and the process repeats. The learned weights are used, but they normally stay fixed.

ILLUSTRATION NEEDED
Current visual aid: inference-pass. Forward-pass pipeline with fixed weights underneath and temporary states above.

Forward Pass
Inference creates temporary states while durable weights stay fixed.

1. Current context enters the model.
2. Fixed weights are used, not updated.
3. Temporary hidden states lead to next-token scores.

Rubric Scores

Accuracy 5/5	Placement 5/5	Relationship 4/5	Prompt/response 5/5	Durable/temp 5/5
Metaphor 4/5	Brain/cognition 4/5	Visual 4/5	Exercise 4/5	Mobile 4/5

inference | forward-pass | input-context | weight | hidden state | logits | sampling

Prompt vs Response

Given context versus generated tokens

Stage: response-generation | Priority: low | Rubric avg: 4.4/5

DEFINITION
Prompt tokens are given to the model; response tokens are generated by the model one at a time and appended to the context.

CORE EXPLANATION
The model does not process the prompt once and then write a whole answer. It repeatedly runs on the current context. At first, the context contains the prompt and any prior conversation or retrieved material. After the model chooses one response token, that token is appended. The next forward pass sees the prompt plus the response so far.

WHERE IT HAPPENS
During inference and response generation.

DURABLE VS TEMPORARY
Prompt and response-so-far tokens are temporary context. They can influence the current run, but they do not normally update weights.

PROMPT VS RESPONSE
This is the distinction itself: given tokens are prompt/context; generated tokens become the response.

WHY IT MATTERS
It prevents the myth that the model writes a full answer all at once.

RELATIONSHIP
After we distinguish prompt from response, we can see how both are represented as tokens.

METAPHOR
Given cards on a table, then new cards added one at a time.

BRAIN METAPHOR
Like hearing a sentence and continuing it word by word.

BRAIN LIMIT
A person may plan, intend, and revise. The model repeatedly samples next tokens from probabilities.

MISCONCEPTION
The model generates the whole response at once.

VISUAL AID
`prompt-response`

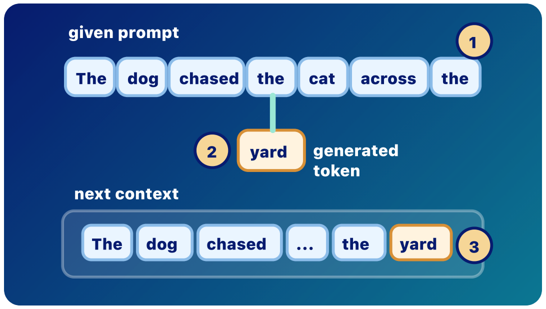
CURRENT EXERCISE
None rendered in Journey; no direct Play mapping.

CHECKPOINT
What happens after one response token is generated? Correct: It is appended to the context. Incorrect: The whole answer appears at once; The model permanently learns it; Softmax disappears.

FEEDBACK
The response grows by next token, append, repeat. Choice notes: The whole answer appears at once: Not quite. A generated token becomes part of the next context, but it is not a durable weight update. The model permanently learns it: Not quite. A generated token becomes part of the next context, but it is not a durable weight update. Softmax disappears: Not quite. A generated token becomes part of the next context, but it is not a durable weight update.

REWRITE NOTES
Risk: The model generates the whole response at once. **Missing:** The model does not process the prompt once and then write a whole answer. It repeatedly runs on the current context. At first, the context contains the prompt and any prior conversation or retrieved material. After the model chooses one response token, that token is appended. The next forward pass sees the prompt plus the response so far.

ILLUSTRATION NEEDED
Current visual aid: prompt-response. Given prompt cards and newly generated response cards entering the next context together.



Prompt vs Response
Prompt tokens are given; response tokens are generated and appended.

1. Prompt tokens are supplied context.
2. The generated token yard is appended.
3. The next run sees prompt plus response so far.

Rubric Scores

Accuracy 5/5	Placement 5/5	Relationship 5/5	Prompt/response 5/5	Durable/temp 4/5
Metaphor 4/5	Brain/cognition 4/5	Visual 4/5	Exercise 4/5	Mobile 4/5

prompt · response · prompt-tokens · response-tokens · generated-token · input-context · completion

Tokenization

Text becomes model-readable chunks

Stage: prompt-processing | **Priority: medium** | **Rubric avg: 4.2/5**

DEFINITION

Tokenization splits text into chunks that can be represented as token IDs.

CORE EXPLANATION

LLMs do not carry raw English sentences through their layers. Text is first split into tokens: pieces that might be whole words, word parts, punctuation, or spaces. Prompt text is tokenized before the model processes it. Generated response tokens are also token IDs before they are turned back into visible text. This app uses simplified token cards for learning; real tokenizers may split differently.

WHERE IT HAPPENS

Before embedding lookup and before transformer layers process the current context.

DURABLE VS TEMPORARY

Tokenization itself does not change model weights. It is a representation step during input/output processing.

PROMPT VS RESPONSE

Both prompt text and generated response text are represented as tokens. Generated response tokens are chosen as token IDs, then displayed as text.

WHY IT MATTERS

Without tokenization, the model has no numerical pieces to process.

RELATIONSHIP

Tokens become token IDs, and token IDs retrieve embeddings.

METAPHOR

Parcels on a conveyor belt entering the model.

BRAIN METAPHOR

Like hearing a sentence in meaningful chunks before interpreting it.

BRAIN LIMIT

Human language perception uses sound, meaning, memory, and experience. Tokenization uses tokenizer rules and token IDs.

MISCONCEPTION

Tokens are always whole words or human concepts.

VISUAL AID

`tokenization`

CURRENT EXERCISE

`prompt-or-response-label`: Prompt or Response?

CHECKPOINT

What can a token be? Correct: A word, word piece, punctuation mark, or other text chunk. Incorrect: Only a complete word; A permanent memory; A whole finished answer.

FEEDBACK

Tokens are model-readable chunks, not always words. Choice notes: Only a complete word: Not quite. Tokens are chunks chosen by a tokenizer, and they do not have to match human word boundaries. A permanent memory: Not quite. Tokens are chunks chosen by a tokenizer, and they do not have to match human word boundaries. A whole finished answer: Not quite. Tokens are chunks chosen by a tokenizer, and they do not have to match human word boundaries.

REWRITE NOTES

Risk: Tokens are always whole words or human concepts. **Missing:** LLMs do not carry raw English sentences through their layers. Text is first split into tokens: pieces that might be whole words, word parts, punctuation, or spaces. Prompt text is tokenized before the model processes it. Generated response tokens are also token IDs before they are turned back into visible text. This app uses simplified token cards for learning; real tokenizers may split differently.

ILLUSTRATION NEEDED

Current visual aid: tokenization. Tokenizer conveyor splitting a short sentence into uneven chunks.

Text to Tokens

Text is split into model-readable chunks before embedding lookup.

1. Tokens can be words, word pieces, punctuation, or spaces.
2. This app uses simplified tokens; real tokenizers may split differently.

Rubric Scores

Accuracy 5/5	**Placement** 5/5	**Relationship** 4/5	**Prompt/response** 5/5	**Durable/temp** 3/5
Metaphor 4/5	**Brain/cognition** 4/5	**Visual** 4/5	**Exercise** 4/5	**Mobile** 4/5

tokenization | **tokenizer** | **token** | **prompt-tokens** | **response-tokens** | **token-id** | **embedding**

Token IDs

Chunks become lookup numbers

Stage: prompt-processing | Priority: low | Rubric avg: 4.1/5

DEFINITION

A token ID is the number assigned to a token so the model can look up its learned starting vector.

CORE EXPLANATION

After tokenization, each token is represented by an integer ID. The ID is not the meaning. It is a lookup key. The embedding table uses that ID to retrieve the token's learned starting vector. During generation, the model also chooses a next token ID before it is displayed as text.

WHERE IT HAPPENS

Between tokenization and embedding lookup.

DURABLE VS TEMPORARY

Token IDs are part of representation during inference. The mapping is fixed by the tokenizer/model setup; using an ID does not update weights.

PROMPT VS RESPONSE

Prompt tokens become token IDs before processing. Generated response tokens are also token IDs before being shown as text and appended to context.

WHY IT MATTERS

Token IDs are the bridge from text chunks to numerical computation.

RELATIONSHIP

Token IDs point to rows in the embedding table.

METAPHOR

A library call number.

BRAIN METAPHOR

A little like recognizing a symbol and activating a familiar association.

BRAIN LIMIT

Humans do not assign fixed integer IDs to every language chunk. The number itself is not understanding.

MISCONCEPTION

The token ID is the meaning.

VISUAL AID

`token-ids`

CURRENT EXERCISE

`prompt-or-response-label`: Prompt or Response?

CHECKPOINT

Why does the model need token IDs? Correct: To look up embeddings. Incorrect: To store a permanent memory; To skip tensors; To guarantee truth.

FEEDBACK

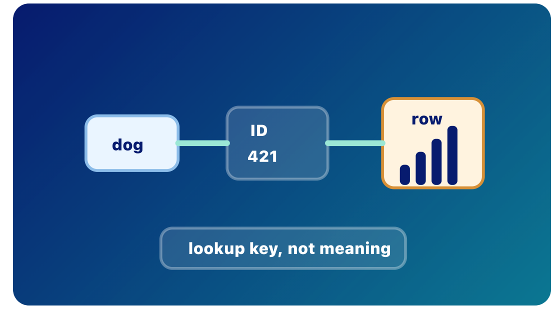
Token IDs connect text chunks to learned embedding vectors. Choice notes: To store a permanent memory: Not quite. A token ID is a lookup key, not meaning, memory, or truth. To skip tensors: Not quite. A token ID is a lookup key, not meaning, memory, or truth. To guarantee truth: Not quite. A token ID is a lookup key, not meaning, memory, or truth.

REWRITE NOTES

Risk: The token ID is the meaning. **Missing:** After tokenization, each token is represented by an integer ID. The ID is not the meaning. It is a lookup key. The embedding table uses that ID to retrieve the token's learned starting vector. During generation, the model also chooses a next token ID before it is displayed as text.

ILLUSTRATION NEEDED

Current visual aid: token-ids. Token chunks mapped to numeric IDs, then to rows in an embedding table.



Token IDs

Each token gets a lookup number that points to an embedding row.

1. The token is not carried forward as raw English.
2. The ID is a lookup key, not the meaning.
3. The ID points to an embedding row.

Rubric Scores

Accuracy 5/5	**Placement** 5/5	**Relationship** 4/5	**Prompt/response** 4/5	**Durable/temp** 3/5
Metaphor 4/5	**Brain/cognition** 4/5	**Visual** 4/5	**Exercise** 4/5	**Mobile** 4/5

token-id | token | tokenizer | embedding-table | embedding

Embeddings

Token IDs look up learned starting vectors

Stage: prompt-processing | Priority: low | Rubric avg: 4.3/5

DEFINITION
An embedding is a learned starting vector retrieved for a token ID.

CORE EXPLANATION
The embedding table is learned during training. At inference time, each token ID in the current context retrieves one row from that table. That row is the token's starting vector before context changes it. Later layers turn embeddings into hidden states that are shaped by the surrounding tokens.

WHERE IT HAPPENS
At the start of model processing, before transformer layers.

DURABLE VS TEMPORARY
The embedding table is durable learned model data. The retrieved embedding vector is used temporarily during a forward pass.

PROMPT VS RESPONSE
Prompt tokens and response-so-far tokens are looked up as embeddings when they are inside the current context. A newly generated token will get its embedding during the next forward pass after it is appended.

WHY IT MATTERS
Embeddings are the first step from token IDs into the model's learned numerical space.

RELATIONSHIP
Embeddings are vectors. The next lesson explains vectors more generally.

METAPHOR
A token gets a starting address in a giant meaning cloud.

BRAIN METAPHOR
Like the first associations that light up when you hear a word.

BRAIN LIMIT
A human association includes memory, sensation, and experience. An embedding is a learned row of numbers.

MISCONCEPTION
An embedding is a dictionary definition or a human memory.

VISUAL AID
`embeddings`

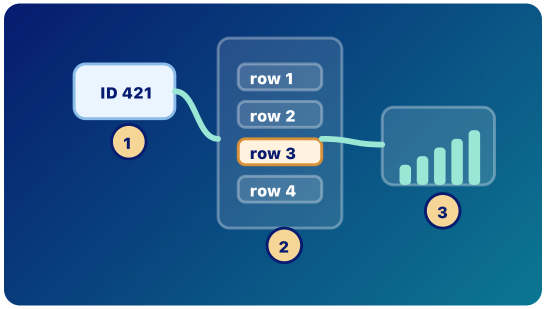
CURRENT EXERCISE
`prompt-or-response-label`: Prompt or Response?

CHECKPOINT
What is an embedding in this app? Correct: A learned starting vector for a token ID. Incorrect: A finished sentence; A human memory; A secret English definition.

FEEDBACK
An embedding is a learned starting vector before context reshapes it. Choice notes: A finished sentence: Not quite. Embeddings are numerical vectors, not definitions, memories, or hidden English text. A human memory: Not quite. Embeddings are numerical vectors, not definitions, memories, or hidden English text. A secret English definition: Not quite. Embeddings are numerical vectors, not definitions, memories, or hidden English text.

REWRITE NOTES
Risk: An embedding is a dictionary definition or a human memory. **Missing:** The embedding table is learned during training. At inference time, each token ID in the current context retrieves one row from that table. That row is the token's starting vector before context changes it. Later layers turn embeddings into hidden states that are shaped by the surrounding tokens.

ILLUSTRATION NEEDED
Current visual aid: embeddings. Lookup table pulling a learned starting vector for one token ID.



Embedding Lookup

A token ID retrieves a learned starting vector.

1. The embedding table was learned during training.
2. Inference retrieves one row temporarily for the current context.
3. Later layers reshape embeddings into hidden states.

Rubric Scores

Accuracy 5/5	**Placement** 5/5	**Relationship** 4/5	**Prompt/response** 4/5	**Durable/temp** 5/5
Metaphor 4/5	**Brain/cognition** 4/5	**Visual** 4/5	**Exercise** 4/5	**Mobile** 4/5

embedding · embedding-table · token-id · vector · hidden state · inference · training

Vectors

Lists of numbers carrying features

Stage: architecture | Priority: high | Rubric avg: 3.8/5

DEFINITION

A vector is a list of numbers that represents features of something, such as a token.

CORE EXPLANATION

In an LLM, many things are represented as vectors. An embedding is a starting vector for a token ID. A hidden state is a later vector shaped by context. The numbers are not usually neat labeled features like dogness or grammar. Features are distributed across many dimensions, which is why vectors can represent fuzzy relationships.

WHERE IT HAPPENS

Throughout the model: embeddings, hidden states, attention calculations, MLP transformations, and output scoring all involve vectors.

DURABLE VS TEMPORARY

Some vectors are durable learned parameters, such as embedding rows. Many vectors during inference are temporary activations, such as hidden states.

PROMPT VS RESPONSE

Prompt tokens and response-so-far tokens become vectors when processed in the current context. Those vectors help produce the next response token.

WHY IT MATTERS

Vectors let the model compute with many fuzzy features at once instead of using plain-English labels.

RELATIONSHIP

Several token vectors organized together become a tensor.

METAPHOR

A dashboard of many sliders.

BRAIN METAPHOR

Like many weak associations around a word lighting up together.

BRAIN LIMIT

The model's vector dimensions are learned numerical patterns, not conscious associations or human-labeled ideas.

MISCONCEPTION

Each vector dimension has a simple human-readable label.

VISUAL AID

`vectors`

CURRENT EXERCISE

None rendered in Journey; no direct Play mapping.

CHECKPOINT

Which statement is most accurate? Correct: Embeddings and hidden states are both vectors. Incorrect: Vectors are whole paragraphs; Vectors are permanent chats; Every vector dimension is a neat English meaning.

FEEDBACK

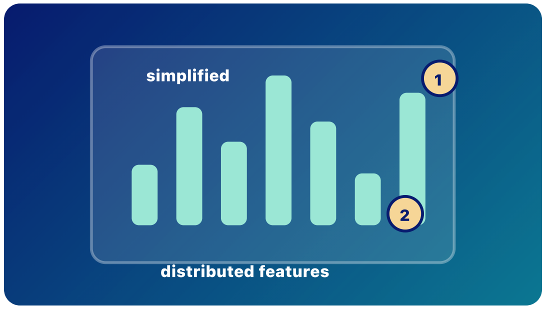
Vectors are lists of numbers used at many points in the model. Choice notes: Vectors are whole paragraphs: Not quite. Vectors are numerical representations; their features are distributed and not usually human-labeled. Vectors are permanent chats: Not quite. Vectors are numerical representations; their features are distributed and not usually human-labeled. Every vector dimension is a neat English meaning: Not quite. Vectors are numerical representations; their features are distributed and not usually human-labeled.

REWRITE NOTES

Risk: Each vector dimension has a simple human-readable label. **Missing:** In an LLM, many things are represented as vectors. An embedding is a starting vector for a token ID. A hidden state is a later vector shaped by context. The numbers are not usually neat labeled features like dogness or grammar. Features are distributed across many dimensions, which is why vectors can represent fuzzy relationships.

ILLUSTRATION NEEDED

Current visual aid: vectors. Many-dimensional feature bars with a warning that labels are simplified.



Feature Vector

A vector is a list of numerical features, not a sentence.

1. Slider labels are simplified teaching examples.
2. Real features are distributed across many dimensions.

Rubric Scores

Accuracy 5/5	Placement 4/5	Relationship 4/5	Prompt/response 3/5	Durable/temp 5/5
Metaphor 4/5	Brain/cognition 4/5	Visual 3/5	Exercise 2/5	Mobile 4/5

vector · feature · distributed-representation · embedding · hidden state · activation

Tensors

Organized blocks of vectors

Stage: architecture | Priority: high | Rubric avg: 3.7/5

DEFINITION

A tensor is an organized block of numbers, often holding many token vectors at once.

CORE EXPLANATION

Once tokens have embeddings, the model needs to carry many token positions and many features together. A simple way to picture this is a table: token positions down one side, feature numbers across the other. With batches, heads, and layers, the shapes get richer. But the core idea is simple: tensors organize vectors so transformer layers can process the current context.

WHERE IT HAPPENS

Tensors carry data through the transformer layers during a forward pass. Model weights are also stored as tensors.

DURABLE VS TEMPORARY

Weight tensors are durable learned parameters. Activation tensors during inference are temporary.

PROMPT VS RESPONSE

The current context tensor may include prompt tokens, retrieved tokens, prior conversation tokens, and response-so-far tokens.

WHY IT MATTERS

The transformer transforms tensors, not raw words.

RELATIONSHIP

Now that current context is in tensor form, transformer layers can apply attention and MLPs. That leads into Workday.

METAPHOR

A stack of spreadsheets carrying token features through the model.

BRAIN METAPHOR

Like keeping many signals active at once.

BRAIN LIMIT

The brain is not literally storing rectangular arrays with token and feature axes.

MISCONCEPTION

A tensor is one word, one number, or a mysterious object.

VISUAL AID

`tensors`

CURRENT EXERCISE

None rendered in Journey; no direct Play mapping.

CHECKPOINT

Which is the better definition of tensor here? Correct: An organized block of numbers. Incorrect: A word in a dictionary; A rule written by a programmer; A permanent memory of the prompt.

FEEDBACK

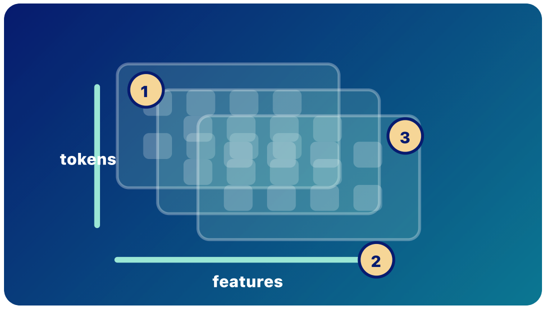
Tensors organize many vectors so the model can compute with them. Choice notes: A word in a dictionary: Not quite. In this app, tensor means a shaped numerical block, not a word, rule, or memory. A rule written by a programmer: Not quite. In this app, tensor means a shaped numerical block, not a word, rule, or memory. A permanent memory of the prompt: Not quite. In this app, tensor means a shaped numerical block, not a word, rule, or memory.

REWRITE NOTES

Risk: A tensor is one word, one number, or a mysterious object. **Missing:** Once tokens have embeddings, the model needs to carry many token positions and many features together. A simple way to picture this is a table: token positions down one side, feature numbers across the other. With batches, heads, and layers, the shapes get richer. But the core idea is simple: tensors organize vectors so transformer layers can process the current context.

ILLUSTRATION NEEDED

Current visual aid: tensors. Mobile-readable tensor block with token-position and feature axes.



Tensor Block

Tensors organize many token vectors for layer-by-layer processing.

1. Token positions run down one axis.
2. Feature dimensions run across another axis.
3. Batch x tokens x features is an advanced shape note.

Rubric Scores

Accuracy 5/5	Placement 4/5	Relationship 4/5	Prompt/response 3/5	Durable/temp 3/5
Metaphor 4/5	Brain/cognition 4/5	Visual 4/5	Exercise 2/5	Mobile 4/5

tensor · vector · activation · weight-tensor · input-context · layer

Attention

Weighted relevance between positions

Stage: architecture | Priority: high | Rubric avg: 4.4/5

DEFINITION
Attention assigns weighted relevance between token positions.

CORE EXPLANATION
Attention assigns weighted relevance between token positions.

WHERE IT HAPPENS
It happens inside transformer layers during a forward pass.

DURABLE VS TEMPORARY
Not listed in current app data.

PROMPT VS RESPONSE
This card explains model machinery; it needs a visible bridge back to how a prompt becomes response tokens.

WHY IT MATTERS
Attention lets one token position use information from other visible token positions in the current context.

RELATIONSHIP
During inference, attention operates over token positions in the current context, including prompt tokens and earlier generated response tokens.

METAPHOR
Spotlights showing which earlier tokens are useful right now.

BRAIN METAPHOR
The word attention is useful because some parts of the context matter more than others.

BRAIN LIMIT
It is not human attention, awareness, desire, or focus. It is weighted relevance in a computation.

MISCONCEPTION
The term attention invites human-focus and awareness metaphors.

VISUAL AID
`attention`

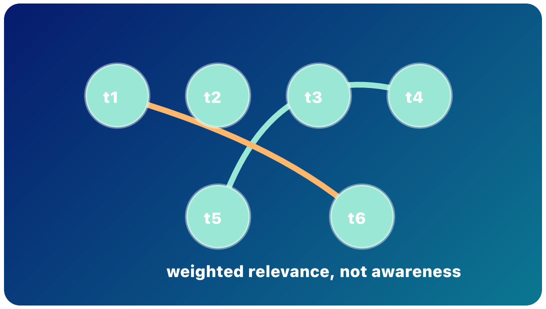
CURRENT EXERCISE
`attention-relevance`: Attention Is Relevance

CHECKPOINT
Which tokens can attention look across during inference? Correct: Tokens currently in the context window. Incorrect: All text the model has ever seen; Only the newest generated token.

FEEDBACK
Attention operates over the current context. It does not browse all training data.

REWRITE NOTES
Risk: The term attention invites human-focus and awareness metaphors. **Missing:** Show attention weights as arrows between positions, not as a mind selecting ideas.

ILLUSTRATION NEEDED
Current visual aid: attention. Token nodes with weighted relevance arcs and a plain-language caption.



Attention Weave
Attention is weighted relevance between token positions.

Rubric Scores

Accuracy 5/5	**Placement** 5/5	**Relationship** 4/5	**Prompt/response** 5/5	**Durable/temp** 4/5
Metaphor 4/5	**Brain/cognition** 5/5	**Visual** 4/5	**Exercise** 4/5	**Mobile** 4/5

attention | input context | prompt tokens | response tokens | hidden state | layer

MLP

Feature reshaping per token

Stage: architecture | Priority: high | Rubric avg: 4.3/5

DEFINITION
An MLP is a feed-forward network that reshapes each token position’s feature vector.

CORE EXPLANATION
An MLP is a feed-forward network that reshapes each token position’s feature vector.

WHERE IT HAPPENS
It appears inside transformer layers, usually after attention has mixed information across positions.

DURABLE VS TEMPORARY
Not listed in current app data.

PROMPT VS RESPONSE
This card explains model machinery; it needs a visible bridge back to how a prompt becomes response tokens.

WHY IT MATTERS
The MLP helps turn mixed context into more useful per-token features for later layers.

RELATIONSHIP
During a forward pass over the current context, attention mixes information across positions and the MLP reshapes features within each position.

METAPHOR
A forge that bends each token vector into a more useful shape.

BRAIN METAPHOR
It is somewhat like a small processing step after noticing relevant context.

BRAIN LIMIT
The MLP is not a neuron with feelings or beliefs; it is a learned function applied to vectors.

MISCONCEPTION
The MLP can blur into attention unless cross-position mixing and per-position reshaping are contrasted.

VISUAL AID
`mlp`

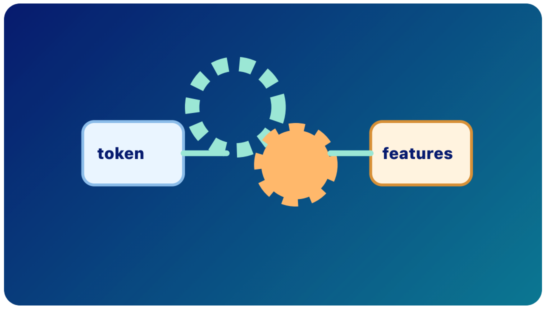
CURRENT EXERCISE
`mlp-feature-reshape`: MLP Feature Reshape

CHECKPOINT
What does the MLP mainly do here? Correct: Reshapes per-token features. Incorrect: Reads private files; Chooses the final answer directly.

FEEDBACK
MLPs transform token feature vectors after attention has mixed contextual information.

REWRITE NOTES
Risk: The MLP can blur into attention unless cross-position mixing and per-position reshaping are contrasted. **Missing:** Put MLP beside attention in the same layer so their complementary roles are visible.

ILLUSTRATION NEEDED
Current visual aid: mlp. Per-token vector forge or feature mixer after attention.



MLP Reshape
The MLP reshapes each token position feature vector.

Rubric Scores

Accuracy 5/5	**Placement** 5/5	**Relationship** 5/5	**Prompt/response** 4/5	**Durable/temp** 4/5
Metaphor 4/5	**Brain/cognition** 4/5	**Visual** 4/5	**Exercise** 4/5	**Mobile** 4/5

MLP | input context | hidden state | attention

Layers

Repeated refinement

Stage: architecture | Priority: medium | Rubric avg: 3.9/5

DEFINITION

A transformer layer is a repeated block that usually combines attention, MLP feature reshaping, residual paths, and normalization.

CORE EXPLANATION

A transformer layer is a repeated block that usually combines attention, MLP feature reshaping, residual paths, and normalization.

WHERE IT HAPPENS

Layers are stacked inside the model between embeddings and next-token scores.

DURABLE VS TEMPORARY

Not listed in current app data.

PROMPT VS RESPONSE

This card explains model machinery; it needs a visible bridge back to how a prompt becomes response tokens.

WHY IT MATTERS

Repeated layers let simple starting vectors become richer, context-shaped hidden states.

RELATIONSHIP

Each layer refines hidden states while residual paths help useful information carry forward.

METAPHOR

A series of editing passes where each pass can revise but also preserve the draft.

BRAIN METAPHOR

The stack can feel like stages of processing.

BRAIN LIMIT

The layers do not form a human-like chain of thought; they update numerical states.

MISCONCEPTION

Layer stacks can be mistaken for a human chain of thought.

VISUAL AID

`layers`

CURRENT EXERCISE

None rendered in Journey; no direct Play mapping.

CHECKPOINT

What do layers repeatedly refine? Correct: Hidden states. Incorrect: Permanent user memory; The app interface.

FEEDBACK

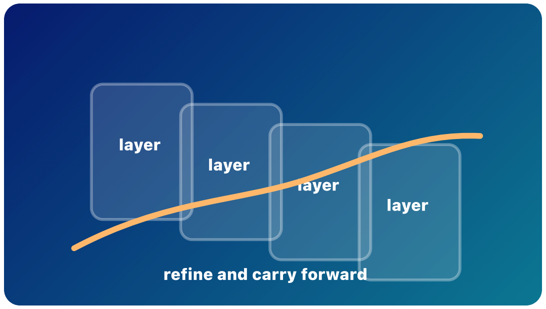
Layers update temporary internal vectors during inference.

REWRITE NOTES

Risk: Layer stacks can be mistaken for a human chain of thought. **Missing:** Add residual paths and normalization as optional advanced labels or a split target lesson.

ILLUSTRATION NEEDED

Current visual aid: layers. Transparent layer stack with attention and MLP blocks repeated.



Layer Stack
Repeated blocks refine hidden states while carrying useful signal forward.

Rubric Scores

Accuracy 5/5	**Placement** 5/5	**Relationship** 4/5	**Prompt/response** 3/5	**Durable/temp** 4/5
Metaphor 4/5	**Brain/cognition** 4/5	**Visual** 4/5	**Exercise** 2/5	**Mobile** 4/5

layer | attention | MLP | hidden state

Hidden States

Temporary contextual vectors

Stage: inference | Priority: high | Rubric avg: 4.4/5

DEFINITION
A hidden state is the model's temporary internal context-shaped vector for a token at a given layer.

CORE EXPLANATION
A hidden state is the model's temporary internal context-shaped vector for a token at a given layer.

WHERE IT HAPPENS
Hidden states appear during the forward pass after embeddings enter the layer stack.

DURABLE VS TEMPORARY
Not listed in current app data.

PROMPT VS RESPONSE
This is the live prompt-to-response path: fixed weights process current context and generate response tokens.

WHY IT MATTERS
They are where the prompt's context changes the token representation before the model scores possible next tokens.

RELATIONSHIP
Hidden states are created while processing the current context during a forward pass; they are not the visible response.

METAPHOR
A scratchpad of numbers that changes while the prompt is being processed.

BRAIN METAPHOR
Hidden state can resemble temporary working memory.

BRAIN LIMIT
It is not permanent memory, consciousness, or private English text; it is a temporary vector.

MISCONCEPTION
Hidden states can be mistaken for secret English text or permanent memory.

VISUAL AID
`hidden-states`

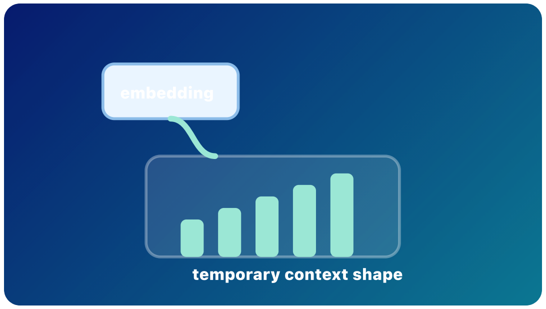
CURRENT EXERCISE
`mlp-feature-reshape`: MLP Feature Reshape

CHECKPOINT
Why is it called hidden state? Correct: It is internal numbers, not visible text. Incorrect: It is encrypted English; It is a secret memory file.

FEEDBACK
Hidden means inside the model's working representation.

REWRITE NOTES
Risk: Hidden states can be mistaken for secret English text or permanent memory. **Missing:** Show hidden state changing layer by layer for the same token in two contexts.

ILLUSTRATION NEEDED
Current visual aid: hidden-states. Same token glowing differently across contexts and layers.



Temporary Hidden State
Hidden states are context-shaped vectors created during a forward pass.

Rubric Scores

Accuracy 5/5	**Placement** 5/5	**Relationship** 4/5	**Prompt/response** 5/5	**Durable/temp** 5/5
Metaphor 4/5	**Brain/cognition** 4/5	**Visual** 4/5	**Exercise** 4/5	**Mobile** 4/5

hidden state · input context · forward pass · embedding · layer · inference

Logits

Raw next-token scores

Stage: response generation | Priority: medium | Rubric avg: 4.4/5

DEFINITION

Logits are raw scores for the next token before they become probabilities.

CORE EXPLANATION

Logits are raw scores for the next token before they become probabilities.

WHERE IT HAPPENS

They appear near the end of a forward pass, after the final hidden state is projected toward the vocabulary.

DURABLE VS TEMPORARY

Not listed in current app data.

PROMPT VS RESPONSE

This card belongs to decoding: the model is choosing one next response token, appending it, then repeating.

WHY IT MATTERS

Logits show that the model has not chosen a token yet; it has scored candidates.

RELATIONSHIP

A forward pass over the current context produces logits for one next generated token.

METAPHOR

A scoreboard before the points are converted into odds.

BRAIN METAPHOR

It can feel like considering options.

BRAIN LIMIT

The model is not weighing options with intent; logits are raw numerical scores.

MISCONCEPTION

Raw scores can be confused with probabilities or truth confidence.

VISUAL AID

`logits`

CURRENT EXERCISE

`softmax-funnel`: Softmax Funnel

CHECKPOINT

Where do probabilities come from in this path? Correct: Softmax converts logits. Incorrect: Training rewrites the weights; The model asks a database.

FEEDBACK

Logits are raw scores; softmax turns them into probabilities.

REWRITE NOTES

Risk: Raw scores can be confused with probabilities or truth confidence. **Missing:** Make vocabulary projection explicit: final hidden state to one score per candidate token.

ILLUSTRATION NEEDED

Current visual aid: logits. Raw candidate scoreboard before softmax.

Raw Scores
Logits are raw next-token scores before probabilities.

Rubric Scores

Accuracy 5/5	Placement 5/5	Relationship 5/5	Prompt/response 5/5	Durable/temp 4/5
Metaphor 4/5	Brain/cognition 4/5	Visual 4/5	Exercise 4/5	Mobile 4/5

logits | generated token | forward pass | hidden state | softmax

Softmax

Scores become probabilities

Stage: response generation | Priority: medium | Rubric avg: 4.4/5

DEFINITION
Softmax turns next-token raw scores into probabilities that sum to one.

CORE EXPLANATION
Softmax turns next-token raw scores into probabilities that sum to one.

WHERE IT HAPPENS
It sits between logits and sampling during a decoding step.

DURABLE VS TEMPORARY
Not listed in current app data.

PROMPT VS RESPONSE
This card belongs to decoding: the model is choosing one next response token, appending it, then repeating.

WHY IT MATTERS
Softmax makes scores comparable as a probability cloud over candidate tokens.

RELATIONSHIP
Softmax converts logits into probabilities; sampling then chooses one token from those probabilities.

METAPHOR
A funnel turning scoreboard points into odds.

BRAIN METAPHOR
It resembles turning vague preferences into relative chances.

BRAIN LIMIT
The probabilities are mathematical outputs, not confidence, desire, or truth.

MISCONCEPTION
Probabilities can be mistaken for correctness or factual confidence.

VISUAL AID
`softmax`

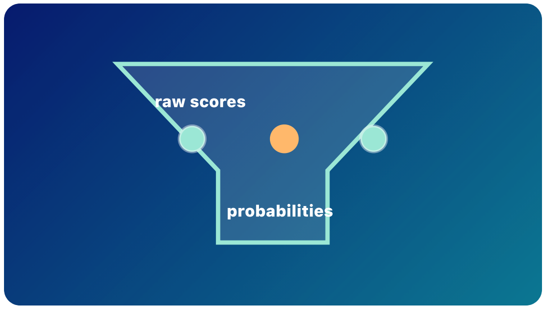
CURRENT EXERCISE
`softmax-funnel`: Softmax Funnel

CHECKPOINT
What does softmax produce? Correct: Next-token probabilities. Incorrect: Permanent memory; A whole essay.

FEEDBACK
Softmax turns raw next-token scores into a probability distribution.

REWRITE NOTES
Risk: Probabilities can be mistaken for correctness or factual confidence. **Missing:** Show that softmax normalizes scores into a distribution that sums to one.

ILLUSTRATION NEEDED
Current visual aid: softmax. Funnel converting raw scores into probabilities that sum to one.



Softmax Funnel
Softmax converts raw scores into probabilities that sum to one.

Rubric Scores

Accuracy 5/5	Placement 5/5	Relationship 5/5	Prompt/response 5/5	Durable/temp 4/5
Metaphor 4/5	Brain/cognition 4/5	Visual 4/5	Exercise 4/5	Mobile 4/5

softmax | logits | decoding step | sampling | probability

Sampling

One next token is chosen

Stage: response generation | Priority: medium | Rubric avg: 4.3/5

DEFINITION
Sampling chooses one next token for the response from the probability cloud.

CORE EXPLANATION
Sampling chooses one next token for the response from the probability cloud.

WHERE IT HAPPENS
It happens after softmax in the decoding step.

DURABLE VS TEMPORARY
Not listed in current app data.

PROMPT VS RESPONSE
This card belongs to decoding: the model is choosing one next response token, appending it, then repeating.

WHY IT MATTERS
Sampling explains why the same prompt can sometimes produce different good responses.

RELATIONSHIP
Temperature, top-k, and top-p shape the candidate pool before the chosen response token is appended to the context.

METAPHOR
Choosing from a weighted bowl of possible next tokens.

BRAIN METAPHOR
It can resemble choosing a word while speaking.

BRAIN LIMIT
The model is not choosing with intention; a decoding rule selects from probabilities.

MISCONCEPTION
Randomness can sound arbitrary unless tied to weighted probabilities.

VISUAL AID
`sampling`

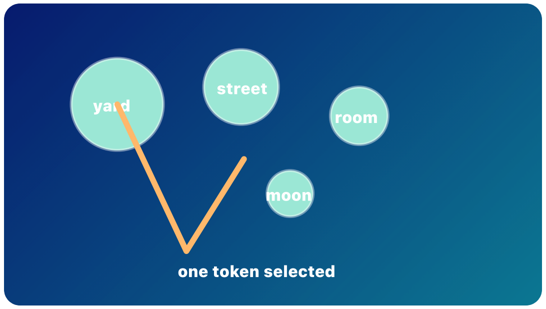
CURRENT EXERCISE
`pick-next-token`: Pick the Next Token

CHECKPOINT
When the model chooses one token, what happens next? Correct: The token is appended and the model runs again. Incorrect: The model writes the whole response at once; The model permanently learns the token.

FEEDBACK
The selected response token becomes part of the context for the next forward pass.

REWRITE NOTES
Risk: Randomness can sound arbitrary unless tied to weighted probabilities. **Missing:** Add temperature/top-p as knobs that shape selection without overexplaining math.

ILLUSTRATION NEEDED
Current visual aid: sampling. Weighted vocabulary cloud with one selected token.



Sample One Token
Sampling chooses one token from the probability cloud.

Rubric Scores

Accuracy 5/5	**Placement** 5/5	**Relationship** 4/5	**Prompt/response** 5/5	**Durable/temp** 4/5
Metaphor 4/5	**Brain/cognition** 4/5	**Visual** 4/5	**Exercise** 4/5	**Mobile** 4/5

sampling · generated token · decoding step · response tokens · temperature · top-k · top-p

Autoregression

One token, append, repeat

Stage: response generation | Priority: low | Rubric avg: 4.4/5

DEFINITION
Autoregression means response generation by repeated next-token prediction: choose one token, append it, and run again.

CORE EXPLANATION
Autoregression means response generation by repeated next-token prediction: choose one token, append it, and run again.

WHERE IT HAPPENS
It is the loop that builds text during inference.

DURABLE VS TEMPORARY
Not listed in current app data.

PROMPT VS RESPONSE
This card belongs to decoding: the model is choosing one next response token, appending it, then repeating.

WHY IT MATTERS
Autoregression explains why earlier generated tokens become part of what the model can use next.

RELATIONSHIP
Each generated response token becomes part of the input context for the next decoding step.

METAPHOR
A train adding one car at a time.

BRAIN METAPHOR
It resembles continuing a thought step by step.

BRAIN LIMIT
The model is not holding a plan in mind; it repeatedly predicts the next token from visible context.

MISCONCEPTION
Learners may still imagine the model drafting a full paragraph internally before showing it.

VISUAL AID
`autoregression`

CURRENT EXERCISE
`prompt-or-response-label`: Prompt or Response?

CHECKPOINT
How does an LLM normally generate text? Correct: One token at a time. Incorrect: All words at once; By rewriting its weights.

FEEDBACK
LLM generation is autoregressive: next token, append, repeat.

REWRITE NOTES
Risk: Learners may still imagine the model drafting a full paragraph internally before showing it. **Missing:** Use one short dog/cat sentence to show next token, append, repeat.

ILLUSTRATION NEEDED
Current visual aid: autoregression. Loop diagram: score, sample, append, run again.

Append and Repeat
The chosen token is appended, then the model runs again.

Rubric Scores

Accuracy 5/5	Placement 5/5	Relationship 5/5	Prompt/response 5/5	Durable/temp 4/5
Metaphor 4/5	Brain/cognition 4/5	Visual 4/5	Exercise 4/5	Mobile 4/5

autoregression · completion · generated token · response tokens · sampling · inference

Context Window

Temporary working context

Stage: current context | Priority: high | Rubric avg: 4.5/5

DEFINITION
A context window is the limited amount of tokens or media the model can currently use.

CORE EXPLANATION
A context window is the limited amount of tokens or media the model can currently use.

WHERE IT HAPPENS
It surrounds the prompt, conversation history, retrieved material, and response-so-far that remain visible.

DURABLE VS TEMPORARY
Not listed in current app data.

PROMPT VS RESPONSE
This card should separate the temporary visible context from durable memory or weight updates.

WHY IT MATTERS
It explains why models can use current context but do not automatically form permanent memory from every chat.

RELATIONSHIP
The model can only use tokens currently inside the context window, whether they came from the prompt, prior conversation, retrieved documents, or the response-so-far.

METAPHOR
A moving spotlight over a growing train.

BRAIN METAPHOR
It is loosely like working memory: what is currently available matters.

BRAIN LIMIT
It is not durable memory. What falls out cannot be directly attended to in the next pass.

MISCONCEPTION
Current context can be confused with saved memory or training data.

VISUAL AID
`context-window`

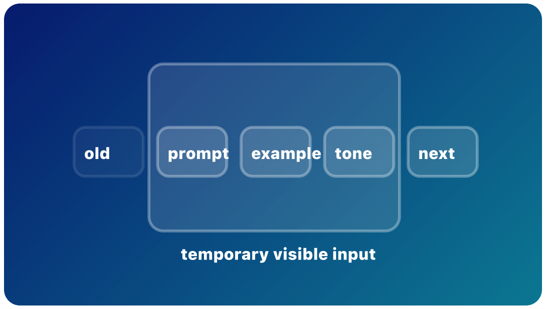
CURRENT EXERCISE
`context-window-fell-out`: Context Window: What Fell Out?

CHECKPOINT
What can happen when the context window is full? Correct: Older tokens may be truncated. Incorrect: The model permanently learns them; Softmax disappears.

FEEDBACK
Context is temporary input, not durable training.

REWRITE NOTES
Risk: Current context can be confused with saved memory or training data. **Missing:** Show what falls out and that retrieval adds text to context rather than training weights.

ILLUSTRATION NEEDED
Current visual aid: context-window. Limited stack of cards where older cards fall out.



Temporary Context Window
Only visible context can influence the next token.

Rubric Scores

Accuracy 5/5	Placement 5/5	Relationship 5/5	Prompt/response 5/5	Durable/temp 5/5
Metaphor 4/5	Brain/cognition 4/5	Visual 4/5	Exercise 4/5	Mobile 4/5

context window · input context · prompt tokens · response tokens · rag · autoregression · inference

RAG and Retrieval

Retrieved text joins the current context

Stage: current context | Priority: high | Rubric avg: 4.6/5

DEFINITION

Retrieval-augmented generation, or RAG, retrieves outside information and places it into the model's context before generating a response.

CORE EXPLANATION

Retrieval-augmented generation, or RAG, retrieves outside information and places it into the model's context before generating a response.

WHERE IT HAPPENS

It happens during inference, before or during response generation, and usually does not change model weights.

DURABLE VS TEMPORARY

Not listed in current app data.

PROMPT VS RESPONSE

Retrieved documents become prompt/context tokens. The answer is still generated as response tokens one at a time.

WHY IT MATTERS

RAG helps learners see that the model is not magically knowing private documents; retrieved snippets become temporary context.

RELATIONSHIP

RAG depends on the context window: retrieved text becomes prompt/context tokens, then the model still uses attention, hidden states, logits, softmax, and sampling to generate response tokens.

METAPHOR

Open-book AI: the model gets notes before answering.

BRAIN METAPHOR

It is a little like looking something up in notes before answering a question.

BRAIN LIMIT

A human can judge sources, remember what was read, and reason about trust in richer ways. The model still generates likely response tokens from retrieved context and learned weights.

MISCONCEPTION

RAG can be mistaken for permanent learning, full file access, web search by itself, fine-tuning, or a guarantee that answers are true.

VISUAL AID

`rag-retrieval`

CURRENT EXERCISE

`open-book-or-learned`: Open Book or Learned?

CHECKPOINT

What does RAG usually do? Correct: Retrieves outside information and places it into the current context. Incorrect: Permanently updates the model's weights; Makes the model conscious of documents; Guarantees that every answer is true.

FEEDBACK

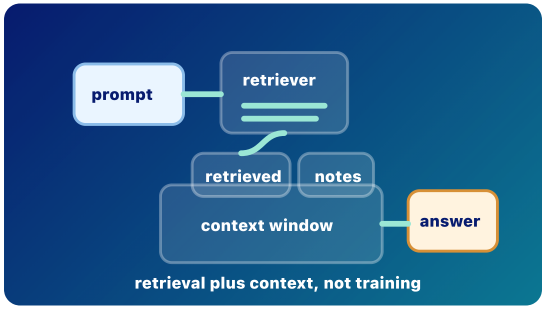
RAG is retrieval plus context. It can improve grounding, but it is not training, fine-tuning, permanent memory, or a truth guarantee. Choice notes: Permanently updates the model's weights: Not quite. Unless a separate training process happens, RAG does not durably update model weights. Makes the model conscious of documents: Not quite. Retrieved documents become visible context; the model does not become aware of them. Guarantees that every answer is true: Not quite. RAG can reduce hallucinations, but retrieval can be poor and the model can still misuse context.

REWRITE NOTES

Risk: RAG can be mistaken for permanent learning, full file access, web search by itself, fine-tuning, or a guarantee that answers are true. **Missing:** Name the retrieval step, the context insertion step, and the fact that weights remain unchanged unless training also happens.

ILLUSTRATION NEEDED

Current visual aid: rag-retrieval. Three lanes: user question, retriever searches a document shelf, retrieved cards enter the context window, and the LLM generates response tokens.



Open-Book Retrieval
Retrieved document cards enter the context before response tokens are generated.

Rubric Scores

Accuracy 5/5	Placement 5/5	Relationship 5/5	Prompt/response 5/5	Durable/temp 5/5
Metaphor 4/5	Brain/cognition 4/5	Visual 5/5	Exercise 4/5	Mobile 4/5

rag · retrieval · grounding · context window · input context · prompt tokens · response tokens · inference · fine-tuning · training · hallucination

How AI Learns

Durable learning versus temporary steering

Stage: risk/policy **Priority: high** **Rubric avg: 3.9/5**

DEFINITION
AI systems can improve or behave differently through durable training, targeted fine-tuning, retrieval, or temporary in-context steering.

CORE EXPLANATION
AI systems can improve or behave differently through durable training, targeted fine-tuning, retrieval, or temporary in-context steering.

WHERE IT HAPPENS
These processes happen at different times: before deployment, between deployments, or inside one current run.

DURABLE VS TEMPORARY
Not listed in current app data.

PROMPT VS RESPONSE
This card uses the prompt/response path to separate practical risk from myths about secret learning or agency.

WHY IT MATTERS
Calling every behavior change learning creates confusion about privacy, memory, and institutional risk.

RELATIONSHIP
Pretraining and fine-tuning can durably change weights; retrieval and examples in the prompt change only what the model can see during inference.

METAPHOR
Revising the textbook versus bringing notes into an open-book exam.

BRAIN METAPHOR
The word learning is useful when prior experience changes later behavior.

BRAIN LIMIT
In-context examples are not personal learning; they steer the current run without normally updating weights.

MISCONCEPTION
Calling every behavior change learning can blur training, retrieval, and in-context steering.

VISUAL AID
`ai-learns`

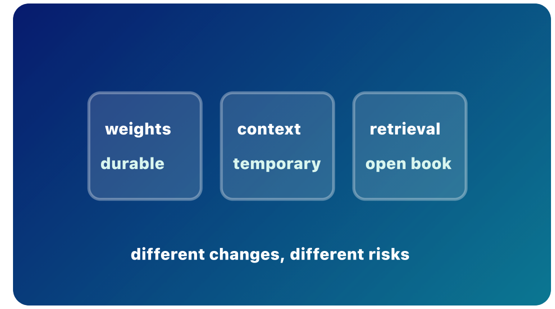
CURRENT EXERCISE
None rendered in Journey; no direct Play mapping.

CHECKPOINT
Which process normally changes only the current context, not weights? Correct: Retrieval-augmented generation. Incorrect: Pretraining; Fine-tuning.

FEEDBACK
RAG places retrieved material into the current context. It does not normally train the base model.

REWRITE NOTES
Risk: Calling every behavior change learning can blur training, retrieval, and in-context steering. **Missing:** Separate durable training, retrieval, temporary instructions, and deployed online learning in one table.

ILLUSTRATION NEEDED
Current visual aid: ai-learns. Matrix of durable weight change, temporary context steering, and retrieval.



Learning Modes
Durable training, retrieval, and temporary steering change different things.

Rubric Scores

Accuracy 4/5	Placement 4/5	Relationship 4/5	Prompt/response 3/5	Durable/temp 5/5
Metaphor 4/5	Brain/cognition 4/5	Visual 4/5	Exercise 4/5	Mobile 3/5

training · pretraining · fine-tuning · inference · rag · in-context learning

Diffusion vs Autoregression

Denoising is a different generation pattern

Stage: architecture | Priority: medium | Rubric avg: 3.6/5

DEFINITION
Diffusion generation usually starts from noise and denoises step by step.

CORE EXPLANATION
Diffusion generation usually starts from noise and denoises step by step.

WHERE IT HAPPENS
Diffusion is common in image generation and other media workflows, not the standard loop for text LLM generation.

DURABLE VS TEMPORARY
Not listed in current app data.

PROMPT VS RESPONSE
This card explains model machinery; it needs a visible bridge back to how a prompt becomes response tokens.

WHY IT MATTERS
The comparison prevents learners from treating all generative AI as the same mechanism.

RELATIONSHIP
Autoregressive LLMs build text token by token; diffusion models refine noise into an image or other output.

METAPHOR
Writing the next word versus developing a photograph out of static.

BRAIN METAPHOR
Both systems can feel imaginative because they produce new outputs.

BRAIN LIMIT
The mechanisms differ: diffusion denoises a representation; autoregressive text predicts and appends tokens.

MISCONCEPTION
Learners may overgeneralize diffusion mechanics to text LLM generation.

VISUAL AID
`diffusion`

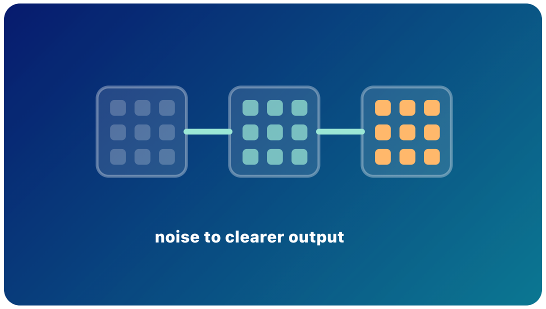
CURRENT EXERCISE
None rendered in Journey; no direct Play mapping.

CHECKPOINT
What does a diffusion model usually start with? Correct: Noise. Incorrect: A completed paragraph; A spreadsheet.

FEEDBACK
Diffusion models learn to denoise toward useful outputs.

REWRITE NOTES
Risk: Learners may overgeneralize diffusion mechanics to text LLM generation. **Missing:** Show a side-by-side: denoise image representation versus append text tokens.

ILLUSTRATION NEEDED
Current visual aid: diffusion. Denoising sequence beside a next-token text loop.



Denoise, Not Append
Diffusion refines noise step by step instead of generating text token by token.

Rubric Scores

Accuracy 5/5	**Placement** 4/5	**Relationship** 4/5	**Prompt/response** 2/5	**Durable/temp** 3/5
Metaphor 4/5	**Brain/cognition** 4/5	**Visual** 4/5	**Exercise** 2/5	**Mobile** 4/5

diffusion | autoregression

Multimodal AI

Multiple media types together

Stage: architecture | **Priority: medium** | **Rubric avg: 3.7/5**

DEFINITION
Multimodal AI can represent or process multiple media types, such as text, images, audio, or video.

CORE EXPLANATION
Multimodal AI can represent or process multiple media types, such as text, images, audio, or video.

WHERE IT HAPPENS
It appears when systems connect encoders, decoders, or shared representations across media types.

DURABLE VS TEMPORARY
Not listed in current app data.

PROMPT VS RESPONSE
This card explains model machinery; it needs a visible bridge back to how a prompt becomes response tokens.

WHY IT MATTERS
It explains how a system can read an image, answer in text, or combine speech and documents.

RELATIONSHIP
Different modalities may use connected encoders, shared vector spaces, or coordinated model components.

METAPHOR
A transit hub where different kinds of information can transfer lines.

BRAIN METAPHOR
People combine sight, speech, and text naturally, so the metaphor helps.

BRAIN LIMIT
Machine modalities are engineered representations, not human sensory experience.

MISCONCEPTION
Human sensory metaphors can make engineered media representations sound more human-like than they are.

VISUAL AID
`multimodal`

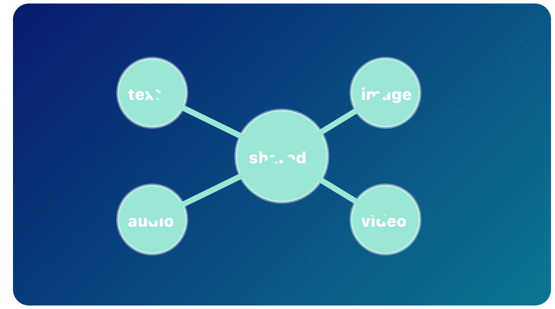
CURRENT EXERCISE
None rendered in Journey; no direct Play mapping.

CHECKPOINT
What does multimodal mean? Correct: Multiple media types. Incorrect: A larger context window only; A model with feelings.

FEEDBACK
Multimodal systems work across media types, often through learned representations.

REWRITE NOTES
Risk: Human sensory metaphors can make engineered media representations sound more human-like than they are. **Missing:** Add an example of image input, text output, and the representation bridge between them.

ILLUSTRATION NEEDED
Current visual aid: multimodal. Text, image, audio, and video paths meeting in a shared representation hub.



Shared Media Hub
Different media types can connect through learned representations.

Rubric Scores

Accuracy 5/5	**Placement** 4/5	**Relationship** 4/5	**Prompt/response** 3/5	**Durable/temp** 3/5
Metaphor 4/5	**Brain/cognition** 4/5	**Visual** 4/5	**Exercise** 2/5	**Mobile** 4/5

multimodal **embedding** **vector**

Risk vs Myth

Practical institutional literacy

Stage: risk/policy | **Priority: high** | **Rubric avg: 4.3/5**

DEFINITION
Risk literacy separates practical harms from magical stories about what models are.

CORE EXPLANATION
Risk literacy separates practical harms from magical stories about what models are.

WHERE IT HAPPENS
It matters whenever people use AI in classrooms, research, administration, health, advising, or IT systems.

DURABLE VS TEMPORARY
Not listed in current app data.

PROMPT VS RESPONSE
This card uses the prompt/response path to separate practical risk from myths about secret learning or agency.

WHY IT MATTERS
Clear mental models help institutions manage privacy, accuracy, bias, over-trust, and security without inventing magical threats.

RELATIONSHIP
Most real risks come from data, outputs, integrations, incentives, and human over-reliance; ordinary inference does not make the model secretly train itself.

METAPHOR
A campus safety map that marks real hazards clearly.

BRAIN METAPHOR
Brain metaphors can make risks feel urgent and memorable.

BRAIN LIMIT
Do not infer consciousness, intent, or secret self-training from fluent text. Keep the mechanism visible.

MISCONCEPTION
Fluent outputs can invite myths about consciousness, intent, or secret self-training.

VISUAL AID
`risk`

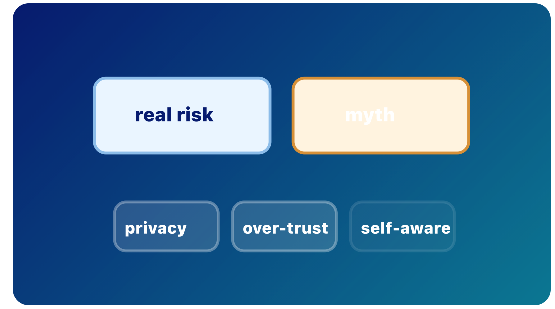
CURRENT EXERCISE
`open-book-or-learned`: Open Book or Learned?

CHECKPOINT
Which is a real institutional risk? Correct: Uploading private data. Incorrect: The model becoming conscious in the chat; Softmax stealing files.

FEEDBACK
Risk literacy is part of model literacy.

REWRITE NOTES
Risk: Fluent outputs can invite myths about consciousness, intent, or secret self-training. **Missing:** Tie each risk to a mechanism: context exposure, tool access, hallucination, bias, or over-trust.

ILLUSTRATION NEEDED
Current visual aid: risk. Risk sorting cards linked to mechanisms rather than fear language.



Risk or Myth
Clear mechanisms help separate practical risk from magical stories.

Rubric Scores

Accuracy 5/5	**Placement** 4/5	**Relationship** 5/5	**Prompt/response** 4/5	**Durable/temp** 5/5
Metaphor 4/5	**Brain/cognition** 4/5	**Visual** 4/5	**Exercise** 4/5	**Mobile** 4/5

inference | **training** | **context window** | **privacy** | **hallucination** | **prompt-injection**